Int. J. Mach. Learn. & Cyber.
DOI 10.1007/s13042-017-0642-3

ORIGINAL ARTICLE

Two swarm intelligence approaches for tuning extreme learning machine

Abobakr Khalil Alshamiri[1] · Alok Singh[1] · Bapi Raju Surampudi[1,2]

Received: 15 April 2016 / Accepted: 18 January 2017
© Springer-Verlag Berlin Heidelberg 2017

Abstract Extreme learning machine (ELM) is a new algorithm for training single-hidden layer feedforward neural networks which provides good performance as well as fast learning speed. ELM tends to produce good generalization performance with large number of hidden neurons as the input weights and hidden neurons biases are randomly initialized and remain unchanged during the learning process, and the output weights are analytically determined. In this paper, two swarm intelligence based metaheuristic techniques, viz. Artificial Bee Colony (ABC) and Invasive Weed Optimization (IWO) are proposed for tuning the input weights and hidden biases. The proposed approaches are called ABC-ELM and IWO-ELM in which the input weights and hidden biases are selected using ABC and IWO respectively and the output weights are computed using the Moore-Penrose (MP) generalized inverse. The proposed approaches are tested on different benchmark classification data sets and simulations show that the proposed approaches obtain good generalization performance in comparison to the other techniques available in the literature.

✉ Alok Singh
alokcs@uohyd.ernet.in

Abobakr Khalil Alshamiri
abobakr2030@yahoo.com

Bapi Raju Surampudi
raju.bapi@iiit.ac.in; bapics@uohyd.ernet.in

1 School of Computer and Information Sciences, University of Hyderabad, Hyderabad 500 046, India

2 Cognitive Science Lab, International Institute of Information Technology, Hyderabad 500 032, India

Keywords Artificial bee colony algorithm · Classification · Extreme learning machine · Invasive weed optimization · Swarm intelligence

1 Introduction

Extreme learning machine (ELM) is a relatively new learning algorithm, proposed by Huang et al. [1, 2], for training single-hidden layer feedforward neural networks (SLFNs). Several variants and improvements of the original ELM have been proposed and applied in various domains and fields [3–6]. In ELM, the input weights and hidden layer biases are randomly generated and the output weights are computed using Moore-Penrose (MP) generalized inverse [7]. Traditional gradient-descent based algorithms, such as back-propagation (BP), for SLFNs require all the weights and biases of the networks to be tuned iteratively. These algorithms usually get stuck in a local minima and suffer from slow convergence. In contrast, ELM provides good generalization performance and converges very fast. In ELM theory, since the input weights and hidden biases are randomly assigned and remained fixed during the learning process, the number of hidden neurons should be large enough in order to obtain good generalization performance [8]. The suitable number of neurons in ELM hidden layer is still an open problem. Several methods have been proposed to determine the suitable number of ELM hidden neurons. Rong et al. [9] proposed a pruned ELM (P-ELM) algorithm for pattern classification problems. The algorithm begins with large number of hidden neurons and then identifies the relevance of the hidden neurons to the class labels using some statistical measures. The hidden neurons which are irrelevant or have low relevance will be eliminated. Miche et al. [10] proposed optimally-pruned ELM (OP-ELM) for

Published online: 03 March 2017

both classification and regression problems. The algorithm starts by constructing a large network using standard ELM Algorithm. Then the multi-response sparse regression algorithm (MRSR) is used to rank the hidden neurons and finally the optimal number of hidden neurons is selected using leave-one-out (LOO) validation. Huang et al. [3] presented incremental extreme learning machine (I-ELM) algorithm in which the neurons are added one-by-one to the hidden layer and the training rate is calculated after adding each neuron. The training will stop when the maximum number of neurons is reached or the training rate is less than the expected one. Lan et al. [11] proposed constructive method for selecting the ELM hidden neurons for regression problems. Their method begins with large number of hidden neurons and then each neuron is selected based on its significance and the selection stops when the unbiased risk estimation based criterion reaches its minimum.

Since ELM requires large number of hidden neurons, there may exist some suboptimal or unnecessary input weights and hidden biases and for some applications, the high number of neurons in the hidden layer renders the ELM respond slowly to unseen testing data [12]. Zhu et al. [12] proposed evolutionary extreme learning machine (E-ELM) algorithm. E-ELM algorithm uses the differential evolution (DE) algorithm to tune the input weights and hidden biases while the output weights are calculated using Moore-Penrose (MP) generalized inverse. In E-ELM, the trial vector is produced by random generation method and the control parameters are manually defined by the user. To improve the performance of E-ELM, Cao et al. [13] proposed a new algorithm named self-adaptive evolutionary extreme learning machine (SaE-ELM) for SLFNs. In SaE-ELM, the ELM hidden layer parameters are optimized by the self-adaptive differential evolution algorithm and the output weights are analytically determined.

Optimizing the input weights and hidden neurons biases is a real-parameter optimization problem. Artificial bee colony (ABC) algorithm [14] and invasive weed optimization algorithm [15] have already proved their effectiveness for real-parameter optimization [16, 17]. This has motivated us to develop two approaches for SLFNs by incorporating the artificial bee colony algorithm and invasive weed optimization algorithm to tune the input weights and hidden neurons biases, and the extreme learning machine to compute the output weights. We refer to these two approaches as ABC-ELM and IWO-ELM. In ABC-ELM, the artificial bee colony algorithm is incorporated to tune the input weights and hidden neurons biases and the extreme learning machine method to analytically determine the output weights. In IWO-ELM, the invasive weed optimization algorithm is used to optimize the input weights and hidden neurons biases and the output weights are calculated using the extreme learning machine method.

It is worth mentioning that while several researchers have used differential evolution (DE) algorithm and its variants for tuning ELM [12, 13], swarm intelligence techniques have not been tried so far. So our paper, to the best of our knowledge, is the first paper to provide an idea about the applicability and viability of swarm intelligence techniques such as ABC and IWO for optimizing ELM hidden parameters. We have also developed two neighboring solution generation procedures. The first procedure is inspired by spatial distribution concept in IWO and the second procedure is based on computing the difference between the current solution and another solution chosen randomly form the population. Both procedures have been used in the proposed ABC-ELM and IWO-ELM approaches. We have also utilized some new data sets to test the performance of the proposed approaches.

The remaining part of this paper is organized as follows: Section 2 presents a brief overview of ELM, ABC and IWO. Section 3 and Section 4 present the proposed ABC-ELM algorithm and IWO-ELM algorithm respectively. Experimental results are presented and analysed in Section 5. Finally, Section 6 concludes this paper.

2 Basic concepts

2.1 Extreme learning machine

ELM is a relatively new learning algorithm for single-hidden layer feedforward neural networks (SLFNs). It randomly initializes network hidden neurons and analytically determines the output weights using Moore-Penrose (MP) generalized inverse [1–3]. Research on randomization of the hidden layer of neural networks started long ago [18–20], but still continues to attract attention. The ELM hidden layer performs a nonlinear transformation of the input patterns into the high dimensional ELM feature space. This transformation often increases the probability of linear separability of the input patterns in the ELM feature space, and hence simplifies the solution of the underlying problems.

Suppose we are given a set of training examples $\aleph = \{(\mathbf{x}_i, \mathbf{t}_i) \mid \mathbf{x}_i \in \mathbf{R}^d, \mathbf{t}_i \in \mathbf{R}^m, i = 1, \ldots, N\}$, then the output of SLFNs with L hidden neurons and activation function $g(x)$ can be represented as [2]:

$$\sum_{i=1}^{L} \beta_i g(\mathbf{w}_i \cdot \mathbf{x}_j + b_i) = \mathbf{y}_j, \;\; j = 1, \ldots, N. \tag{1}$$

where $\mathbf{w}_i = [w_{i1}, \ldots, w_{id}]^T$ and b_i are the input weight and the bias of the hidden neuron, respectively. $\beta_i = [\beta_{i1}, \ldots, \beta_{im}]^T$ is the output weight. $\mathbf{w}_i \cdot \mathbf{x}_j$ is the inner

product of $\mathbf{w}_i$ and $\mathbf{x}_j$. With that standard SLFNs, the parameters $\beta_i, i = 1, \ldots, L$ can be estimated such that

$$\sum_{i=1}^{L} \beta_i g(\mathbf{w}_i \cdot \mathbf{x}_j + b_i) = \mathbf{t}_j,\ j = 1, \ldots, N. \tag{2}$$

Equation (2) can be written as in [2]:

$$\mathbf{H}\boldsymbol{\beta} = \mathbf{T} \tag{3}$$

where

$$\mathbf{H} = \begin{bmatrix} g(\mathbf{w}_1 \cdot \mathbf{x}_1 + b_1) & \ldots & g(\mathbf{w}_L \cdot \mathbf{x}_1 + b_L) \\ \vdots & \ldots & \vdots \\ g(\mathbf{w}_1 \cdot \mathbf{x}_N + b_1) & \ldots & g(\mathbf{w}_L \cdot \mathbf{x}_N + b_L) \end{bmatrix}_{N \times L} \tag{4}$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_1^T \\ \vdots \\ \beta_L^T \end{bmatrix}_{L \times m} \quad \text{and} \quad \mathbf{T} = \begin{bmatrix} \mathbf{t}_1^T \\ \vdots \\ \mathbf{t}_N^T \end{bmatrix}_{N \times m} \tag{5}$$

$\mathbf{H}$ is the hidden layer output matrix of the network [21]. The output weights $\boldsymbol{\beta}$ can be computed analytically by the minimum norm least-square solution:

$$\boldsymbol{\beta} = \mathbf{H}^{\dagger}\mathbf{T} \tag{6}$$

where $\mathbf{H}^{\dagger}$ is the Moore-Penrose generalized inverse of $\mathbf{H}$ [7].

2.2 Artificial bee colony algorithm

The artificial bee colony (ABC) algorithm is a relatively new swarm intelligence based meatheuristic technique. It is proposed by Karaboga [14] in 2005. The ABC algorithm simulates the collective intelligent behavior of honey bee swarm while foraging. There are three groups of foraging bees in the colony of honey bees, viz. employed, onlooker and scout bees. Bees currently associated with food sources are called employed bees. The task of these bees is to bring the loads of nectar from the food sources to the hive and share information about their food sources with other bees waiting in the hive. The information includes the location of the food sources and the amount of nectar. The employed bees share this information by performing a dance in a special area of the hive called dance area. Onlooker bees receive the information about the food sources by watching the dances of the employed bees in the dancing area. Onlooker bees have tendency to choose a food source with probability proportional to its nectar amount. That's the food sources with rich nectar amount will attract more onlookers. Once the onlooker bee chooses a food source to exploit, it becomes employed. Scout bees are randomly searching for new food sources in the neighborhood of the hive. Whenever the food source is fully exhausted, the employed bee associated with that food source abandons it and becomes onlooker or scout. Therefore, the exploration of food sources is performed by scout bees whereas the exploitation is conducted by employed and onlooker bees. The collective knowledge obtained in honey bee colony is due to the exchange of information among bees. This intelligent foraging behavior of honey bees inspired Karaboga [14] to propose the ABC algorithm which mimics the behavior of honey bees while searching for food sources. In ABC algorithm, the artificial bees are categorized into employed, onlooker and scout bees. The functions of the artificial bees are analogous to their real counterparts. Each food source in ABC algorithm is a possible solution to the problem under investigation and the nectar amount of a food source represents the quality (fitness) of the solution represented by that food source. The ABC algorithm assigns only one employed bee for every food source. Therefore, the number of employed bees is the same as the number of food sources [14, 22]. The number of the onlooker bees could be the same as or different from the number of employed bees [23]. In ABC algorithm, whenever a food source is completely depleted, the employed bee exploiting that food source becomes scout. The random search for food sources performed by scout in real honey bees colony is simulated in ABC algorithm by generating a new food source (solution) randomly and associating the scout with this newly generated food source. Once the scout is assigned to the newly generated food source, it becomes employed again.

ABC algorithm follows an iterative process. It begins by generating initial solutions (food sources) randomly and associating these food sources with employed bees. There is only one employed bee for every food source. After initialization, the algorithm improves the initial solutions by performing three search processes iteratively. These processes are employed bees process, onlooker bees process and scout bees process. A simple pseudo-code of the ABC algorithm is shown in Algorithm 1.

Employed bees process: Each employed bee searches for a new food source in the neighborhood of the current food source. The employed bee evaluates the nectar amount (fitness) of the new food source (new solution). If the new food source is better than the old one, the old food source is replaced with new food source. The actual method for obtaining a new food source in the neighborhood of the current food source depends on the problem under consideration. At the end of this process, employed bees share the information about their associated food sources with the onlooker bees.

Onlooker bees process: Based on the shared information, each onlooker bee stochastically chooses a food source to exploit. That's the probability of a food source being selected by the onlooker bees increases with the increase in quality (fitness) of that food source. This results in more onlooker bees assigned to good food sources.

Similar to employed bees, each onlooker bee obtains a new food source in the vicinity of the current food source and evaluate its fitness. If the new food source has better fitness than the old source, onlooker bee abandons the old one and memorizes the new food source.

Scout bees process: If the employed bees and onlooker bees cannot improve the quality of a food source after performing a certain number of attempts *limit*, the food source is considered to be depleted and the employed bee exploiting that food source becomes scout. A new food source (solution) is randomly generated in the same manner as an initial solution. The scout bee is then assigned to this random food source and becomes employed. The ABC algorithm repeats these three processes until the termination condition is satisfied.

Algorithm 1: The artificial bee colony algorithm

```
Generate initial solutions randomly;
Evaluate the initial solutions;
repeat
    Employed bees process;
    Onlooker bees process;
    Scout bees process;
    Save the best solution obtained so far;
until termination condition is satisfied;
```

2.3 Invasive weed optimization algorithm

Invasive weed optimization algorithm (IWO) is a recently proposed swarm intelligence technique which mimics the colonizing behavior of weeds. It is proposed by by Mehrabian and Lucas [15] in 2006. The rapid growth and reproduction of these weeds pose a threat to the cultivated plants. Weeds have great surviving capability due its ability to adapt to the change in environment. In IWO algorithm, each weed represents a possible solution to the problem to be optimized and its fitness corresponds to the fitness of the solution represented by that weed. The IWO algorithm is an iterative algorithm. It starts with a finite number of weeds (solutions) being randomly generated and dispread over the search area. Then the fitness of each weed is evaluated. After initialization, the algorithm tries to find the optimal solution by subjecting the population of solutions to repeated cycles of three search steps, viz. reproduction, spatial dispersal and competitive exclusion. In every iteration, the weeds produce seeds (child solutions) depending on their fitness. The produced seeds are randomly distributed over the search area, usually around the producing weeds, by normally distributed random numbers with mean equal to zero and varying variance. The process of reproduction and spatial dispersal continues until the maximum number of weeds is reached. Once the maximum number of weeds is reached, only the fittest weeds are allowed to produce and the weeds with least fitness are eliminated (competitive exclusion). The reproduction, spatial dispersal and competitive exclusion steps are repeated until the termination condition is satisfied.

The steps of IWO algorithm are explained in detail as follows:

Step 1: *Population Initialization*—A set of initial solutions are randomly generated and dispread over the search space.

Step 2: *Reproduction*—The number of seeds a weed can produce depends on its own, the highest and lowest fitness of the colony. The higher the fitness of the weed is, the more seeds it produces. The number of seeds produced by a weed is calculated as follows [24]:

$$N_{seed} = \frac{fit - fit_l}{fit_h - fit_l}\left(S_{max} - S_{min}\right) + S_{min} \tag{7}$$

where fit is the fitness of the current weed. fit_h and fit_l correspond to the highest and lowest fitness of the population, respectively. S_{max} and S_{min} are respectively the maximum and the minimum number of seeds a weed can produce.

Step 3: *Spatial dispersal*—The produced seeds are randomly distributed over the search area, usually around the producing weeds, by normally distributed random numbers with zero mean and varying variance. The standard deviation (σ) at every iteration will reduced from a previously defined initial value ($\sigma_{initial}$) to a final value (σ_{final}) and is given by the following expression:

$$\sigma_{iter} = \left(\frac{iter_{max} - iter}{iter_{max}}\right)^{nmi}\left(\sigma_{initial} - \sigma_{final}\right) + \sigma_{final} \tag{8}$$

where σ_{iter} is the standard deviation at the current iteration. $iter_{max}$ is the maximum number of iterations and nmi is the nonlinear modulation index. Usually, nmi is set to 3.

Step 4: *Competitive exclusion*—Initially, all weeds will reproduce fast and all seeds (newly produced weeds) will be included in the population until the maximum number of weeds WN_{max} in the colony is reached. Once the maximum number of weeds is reached, only the fittest weeds are allowed to replicate and survive to the next generation and the weeds with least fitness are eliminated.

3 Proposed ABC-ELM algorithm

Due to the random initialization of input weights and hidden biases of ELM network, ELM requires a high number of neurons. Some of the input weights and hidden neurons biases could be unnecessary or suboptimal. And for some applications, the high number of hidden neurons may render ELM slow in responding to unknown testing data.

In this section, we have proposed a hybrid approach named ABC-ELM which uses ABC algorithm to tune the input weights and hidden neurons biases and MP generalized inverse to determine the output weights.

3.1 Initialization

The ABC algorithm randomly generates the population of SN solutions (food source positions). Each solution S includes all input weights and hidden biases:

$$\mathbf{S} = [w_{11}, w_{12}, \ldots, w_{1L}, w_{21}, w_{22}, \ldots, w_{2L}, \ldots, w_{d1}, w_{d2}, \ldots, w_{dL}, b_1, b_2, \ldots, b_L] \tag{9}$$

where the dimension of each solution $\mathbf{S}$ is $D = (d+1) \times L$ and all w_{ij} and b_j are randomly generated within a specfic range (e.g $[-1, 1]$).

3.2 Solution fitness

After initializing all solutions, the corresponding output weights for each solution are calculated by using Eq. (6). For classification problems, the objective is to minimize the number of misclassified patterns. The fitness of the ith solution in the population is computed as follows:

$$fit_i = \frac{1}{1 + MC_i} \tag{10}$$

where MC_i is the number of misclassified patterns obtained by using the solution i. The fitness function value needs to be maximized.

3.3 Generation of neighboring solution

In order to obtain a new solution $\mathbf{V}$ in the neighborhood of the current solution $\mathbf{S}$, we have used two different neighborhood procedures. The first procedure is fitness-based while the second one is iteration-based. The first procedure is inspired by spatial distribution concept in IWO in which the new solutions are generated in the neighborhood of the current solution with zero mean but varying variance. To produce a candidate solution $\mathbf{V}$ from the current solution $\mathbf{S}$, the first procedure uses the following expression:

$$\mathbf{V}_{ij} = \mathbf{S}_{ij} + rand(0, \sigma_{iter}) \tag{11}$$

where $i = 1, 2, \ldots, SN$ and $j = 1, 2, \ldots, D$. $rand$ is a function that returns normally distributed random number with zero mean and standard deviation σ_{iter}.

From Eq. (11), if we modify all elements of the solution $\mathbf{S}$ in order to produce a new solution $\mathbf{V}$, the algorithm may get stuck in a local minima due to premature convergence. In the other hand, if we modify only one element, the algorithm will suffer from slow convergence. In the first procedure, we suggest that the amount of changes/modifications should be based on the fitness of the solutions and is calculated as follows:

$$\begin{aligned} Change &= \frac{D \times ChgPerct}{100} \\ ChgPerct &= \frac{fit_h - fit}{fit_h}\left(Chg_{max} - Chg_{min}\right) + Chg_{min} \end{aligned} \tag{12}$$

where fit is the fitness of the current solution. fit_h corresponds to the highest fitness of the population. Chg_{max} and Chg_{min} are respectively the maximum and the minimum percentage of the current solution elements that can be changed to produce a new solution.

Eq. (12) suggests that the fittest solutions in the population should not perturbed much in order to obtain the neighboring solutions. The steps of generating neighboring solution using the first procedure are given in Algorithm 2

Algorithm 2: The First neighborhood procedure

```
Let S_i = {s_{i,1}, s_{i,2}, ..., s_{i,D}} be current solution and V_i is its neighbor;
Set V = S;
Calculate the value of the variable Change using Eq. (12);
for k = 1 to Change do
    Generate an integer number j in the interval [1, D];
    V_ij = S_ij + rand(0, σ_iter);
```

The second procedure is based on the difference between the parameters/elements of the current solution and another solution chosen randomly from the population. To produce a candidate solution $\mathbf{V}$ from the current solution $\mathbf{S}$, the second procedure uses the following expression:

$$\mathbf{V}_{ij} = \mathbf{S}_{ij} + \phi_{ij}(\mathbf{S}_{ij} - \mathbf{S}_{kj}) \tag{13}$$

where $i, k \in \{1, 2, \ldots, SN\}$, $i \neq k$ and $j = 1, 2, \ldots, D$. ϕ_{ij} is a random number between [−1, 1] which controls the production of the neighboring solution around the current solution.

In second procedure, the elements of the current solution are copied to the neighboring solution with probability P_{copy}. The neighboring solutions will be generated by varying the value of P_{copy} according to the number of iterations. The P_{copy} varies from initial value, $P_{initial}$ to final value, P_{final}. The value of P_{copy} at iteration *iter* is calculated as follows [23]:

$$P_{copy} = \frac{P_{final} - P_{initial}}{iter_{max}}(iter) + P_{initial} \tag{14}$$

where $iter_{max}$ is the maximum number of iterations. From Eq. (14), the probability of copying the elements of the current solution to the neighboring solution increases as the algorithm proceeds. The steps of generating neighboring solution using second procedure are given in Algorithm 3

Algorithm 3: The Second neighborhood procedure

```
Let S_i = {s_{i,1}, s_{i,2}, ..., s_{i,D}} be current solution and V_i is its neighbor;
Calculate the value of the variable P_copy using Eq. (14);
Generate an integer number k in the interval [1, SN];
for j = 1 to D do
    Generate a random number r in the interval [0, 1];
    if r < P_copy then
        Set V_ij = S_ij;
    else
        Set V_ij = S_ij + φ_ij(S_ij − S_kj);
```

In both neighborhood procedures, if the produced parameter/element exceeds its predetermined limit, the value of the parameter will be set to its limit value.

3.4 ABC-ELM algorithm

The ABC-ELM algorithm starts by randomly generating the population of *SN* solutions according to the encoding specified in Eq. (9) and then allocating the employed bees to the the initial solutions. After initializing all solutions, the corresponding output weights for each solution are first calculated by using Eq. (6) and then the fitness of each solution is evaluated using Eq. (10).

After assigning the employed bees to the initial solutions, every employed bee produces its neighbor (new) solution using Algorithm 2 or Algorithm 3. The employed bee will replace the old solution if the fitness of the new solution is better than that of the old solution. The employed bees share the fitness information of their solutions with the onlooker bees. Based on the shared information, each onlooker bee tends to choose a solution with a probability proportional to its fitness value. The probability of selecting a solution *i* is computed as follows:

$$p_i = \frac{fit_i}{\sum_{j=1}^{SN} fit_j} \tag{15}$$

Similar to the employed bees, each onlooker bee produces a new solution, compares its fitness with the fitness of the old solution and chooses a solution with better fitness. The solution of which the fitness value is not improved further after performing a certain number of attempts *limit* is abandoned. The employed bee associated with this not improving solution becomes a scout. The scout bee is then assigned to a random solution generated by using Eq. (9). The ABC algorithm avoids suboptimal solutions through abandoning and replacing not improving solutions. The pseudo-code of the ABC-ELM algorithm is given in Algorithm 4. The version of ABC-ELM algorithm which uses Algorithm 2 to generate the neighboring solution will be referred to as ABC(I)-ELM, whereas the version which uses Algorithm 3 will be referred to as ABC(II)-ELM.

Algorithm 4: The pseudo-code of the ABC-ELM algorithm

```
Given a data set ℵ = {(x_i) | x_i ∈ R^d, i = 1,...,N};
Construct ELM network with L hidden neurons and activation function g(x);
Initialize the population of SN solutions S_ij, i = 1,2,...,SN, j = 1,2,...,D;
Compute the output weights for each solution S_i using Eq. (6);
Evaluate the fitness fit(S_i) of the population S_i using Eq. (10);
Set trial_i = 0, i = 1,2,...,SN;
repeat
    for i = 1 to SN do
        Produce V_i from S_i using Algorithm 2 or Algorithm 3;
        Calculate the fitness value of V_i using Eq. (10);
        if fit(V_i) > fit(S_i) then
            Replace S_i with V_i and set trial_i = 0;
        else
            trial_i = trial_i + 1;
    Compute the probability values p_i for the solutions using Eq. (15);
    for each onlooker bee do
        Choose a solution S_i depending on p_i;
        Produce V_i from S_i using Algorithm 2 or Algorithm 3;
        Calculate the fitness value of V_i using Eq. (10);
        if fit(V_i) > fit(S_i) then
            Replace S_i with V_i and set trial_i = 0;
        else
            trial_i = trial_i + 1;
    for each solution S_i do
        if trial_i > limit then
            Replace S_i with a random solution;
            Set trial_i = 0;
    Save the best solution obtained so far;
until termination condition is satisfied;
```

4 Proposed IWO-ELM algorithm

Similar to ABC-ELM algorithm, The IWO-ELM algoritm uses the same solution encoding specified in Eq. (9). The IWO-ELM algorithm starts with a set of initial weed solutions (WN_{init}) randomly generated and dispread over the search space. It computes the output weights for all solutions by using Eq. (6) and then evaluates the fitness of each weed solution using Eq. (10). Each weed solution produces certain number of seed solutions depending on its own, the highest and lowest fitness of the colony. The number of seeds a weed can produce is calculated by using Eq. (7). Each seed will be produced in same way as the neighboring solution generated in ABC-ELM algorithm. In other words, the seed solution will be produced by using Algorithm 2 or Algorithm 3 where the generating weed represents the current solution and the seed represents the neighboring solution. The seed solutions will be included in the population and their fitnesses are evaluated. This process will continue in every iteration until the maximum number of weed solutions WN_{max} in the colony is reached. Once the maximum number of weed solutions is reached, a competitive exclusion process is performed to eliminate the solutions with poor fitness. In competitive exclusion process, the weeds and the newly generated seed solutions are together sorted according to their fitness. Next, only the fittest weeds are allowed to replicate and survive to the next generation and the weeds with poor fitness are eliminated to maintain the maximum allowable number of solutions (WN_{max}) in a colony. The pseudo-code of the IWO-ELM algorithm is given in Algorithm 5, where WN_c represents the total number of weeds in the colony at any instance of time. The version of IWO-ELM algorithm which uses Algorithm 2 to generate the neighboring solution will be referred to as IWO(I)-ELM, whereas the version which uses Algorithm 3 will be referred to as IWO(II)-ELM.

Algorithm 5: The pseudo-code of the IWO-ELM algorithm

```
Given a data set ℵ = {(x_i) | x_i ∈ R^d, i = 1,...,N};
Construct ELM network with L hidden neurons and activation function g(x);
Initialize the population of WN_init weed solutions S_ij, i = 1,2,...,WN_init, j = 1,2,...,D;
Compute the output weights for each solution S_i using Eq. (6);
Evaluate the fitness fit(S_i) of the population S_i using Eq. (10);
Set WN_c = WN_init;
Sort the weeds WN_c according to their fitness;
if WN_c > WN_max then
    Set WN_c = WN_max;
repeat
    Set WN_colony = WN_c;
    for i = 1 to WN_colony do
        Calculate the number of seeds N_seed for solution S_i using Eq. (7);
        for j = 1 to N_seed do
            Set WN_c = WN_c + 1;
            Generate seed solution S_j from solution S_i using Algorithm 2 or Algorithm 3;
    Sort the weeds WN_c according to their fitness;
    Memorize the best solution achieved so far;
    if WN_c > WN_max then
        Set WN_c = WN_max;
until termination condition is satisfied;
```

Table 1 Specifications of benchmark classification data sets used in [12, 13]

Data sets	Data		Attributes	Classes
	Training	Testing		
Diabetes	500	268	8	2
Disease	100	170	13	2
Iris	100	50	4	3
Wine	100	78	13	3
Image segmentation	1900	410	18	7
Satellite image	5400	1000	36	7
Shuttle	50,750	7250	9	7

Table 2 Specifications of additional benchmark classification data sets used to test our approaches

Data sets	Data		Attributes	Classes
	Training	Testing		
Cardiotocography-3	1595	531	21	3
Cardiotocography-10	1595	531	21	10
Phishing websites	1842	641	30	2
Vehicle	634	212	18	4
Wine quality	1199	400	11	6

5 Experimental study

In this paper, we have tested and compared the performance of the proposed ABC-ELM and IWO-ELM algorithms with the techniques given in [12, 13] on seven benchmark classification data sets used in [12, 13]. The specification of these data sets are given in Table 1. In addition, we have used five more benchmark classification data sets to evaluate the performance of our two algorithms. The specifications of these five data sets are given in Table 2. Hence, the performance of our two algorithms are evaluated on 12 data sets in total. All these data sets can be downloaded from the UCI machine learning repository[1].

5.1 Data sets

The data sets used in this paper are described briefly as follows: The Pima Indians Diabetes data set (Diabetes) is used to diagnose whether the patient is diabetes positive or not. It consists of 768 patterns from two different classes. The heart Disease data set (Disease) consists of 13 attributes and two classes. The class refers to the presence of heart disease in the patient. The Fisher Iris data set

[1] http://www.ics.uci.edu/~mlearn/MLRepository.html

Table 3 Comparison of classification performance of the techniques given in [13] and the proposed ABC-ELM and IWO-ELM algorithms

Data sets	Methods	Testing (%)		Training time (s)	Neurons/SVs
		Accuracy	Dev		
Diabetes	SaE-ELM	79.55	2.68	5.6422	20
	LM	74.73	3.2	2.8978	20
	CFWNN-ELM	78.02	2.56	0.0102	35
	SVM	77.31	2.73	364.6	62.28
	E-ELM^{best}	78.09	3.53	1.475	30
	E-ELM^{average}	77	3.4	1.2491	23
	E-ELM^{worst}	76.06	3.52	1.1754	22
	DE-LM^{best}	77.5	2.08	4.6297	2
	DE-LM^{average}	76.54	2.41	5.0405	2.25
	DE-LM^{worst}	75.94	2.97	4.3526	2
	ABC(I)-ELM	77.13	2.02	4.3326	10
	ABC(II)-ELM	77.19	1.89	4.4723	13
	IWO(I)-ELM	77.03	1.68	7.024	9
	IWO(II)-ELM	77.31	1.69	8.5576	13
Disease	SaE-ELM	82.53	3.86	0.7086	18
	LM	71.75	6.67	0.3066	20
	CFWNN-ELM	76.85	3.43	0.0016	30
	SVM	76.1	3.46	6.7094	81.6
	E-ELM^{best}	81.62	4.29	0.2247	16
	E-ELM^{average}	81.01	3.67	0.2425	15
	E-ELM^{worst}	80.05	3.44	0.2588	14
	DE-LM^{best}	79.06	2.28	2.8203	2
	DE-LM^{average}	77.99	3.06	2.8418	2
	DE-LM^{worst}	77.29	3.13	2.7469	2
	ABC(I)-ELM	81.35	2.39	1.1965	7
	ABC(II)-ELM	81.49	2.56	1.0568	8
	IWO(I)-ELM	81.61	2.81	2.6548	11
	IWO(II)-ELM	81.92	2.41	2.8829	15
Iris	SaE-ELM	97.2	4.12	0.2289	18
	LM	95.4	3.19	0.3641	10
	CFWNN-ELM	95.92	3.05	0.0009	20
	SVM	94.36	2.76	4.5488	23.3
	E-ELM^{best}	96.48	3.67	0.1522	16
	E-ELM^{average}	96.17	3.48	0.1596	16.5
	E-ELM^{worst}	95.8	3.78	0.15	16
	DE-LM^{best}	96.94	2.21	1.2546	6
	DE-LM^{average}	96.29	2.39	1.3177	5.25
	DE-LM^{worst}	95.87	2.2	1.3566	6
	ABC(I)-ELM	97.2	2.46	1.6041	17
	ABC(II)-ELM	97.08	2.39	1.2091	12
	IWO(I)-ELM	97.04	2.40	3.4738	18
	IWO(II)-ELM	97.6	2.21	3.1772	17
Wine	SaE-ELM	97.95	2.54	0.305	22
	LM	92.97	4.36	0.4988	20
	CFWNN-ELM	95.15	3.13	0.0028	30
	SVM	97.48	1.57	5.8941	47.3
	E-ELM^{best}	97.28	2.71	0.2319	22
	E-ELM^{average}	96.85	2.65	0.2608	23.25
	E-ELM^{worst}	96.09	2.88	0.3214	30

Table 3 (continued)

Data sets	Methods	Testing (%)		Training time (s)	Neurons/SVs
		Accuracy	Dev		
	DE-LM^{best}	97.11	1.65	1.9985	4
	DE-LM^{average}	96.59	1.96	1.8947	4.25
	DE-LM^{worst}	96.03	1.99	2.2023	6
	ABC(I)-ELM	97.79	1.72	1.8322	19
	ABC(II)-ELM	97.97	1.97	1.6536	20
	IWO(I)-ELM	98	2.07	3.3855	15
	IWO(II)-ELM	98.13	1.64	3.6714	20

Table 4 Comparison of classification performance of the techniques given in [12] and the proposed ABC-ELM and IWO-ELM algorithms

Data sets	Methods	Testing (%)		Training time (s)	Neurons
		Accuracy	Dev		
Image segmentation	E-ELM	95.27	1.56	154.1	70
	ELM	95.07	1.14	0.1024	200
	DE-LM	88.65	2.05	13196.6	80
	LM	86.27	1.8	4745.7	100
	GALS	94.27	1.95	3423.7	80
	ABC(I)-ELM	95.36	1.25	168.558	170
	ABC(II)-ELM	95.33	1.23	167.637	170
	IWO(I)-ELM	95.64	1.28	172.608	150
	IWO(II)-ELM	95.25	1.26	378.3	170
Satellite image	E-ELM	88.46	1.36	1569.27	90
	ELM	88.99	1.01	1.2065	500
	DE-LM	85.31	2.14	36924	80
	LM	82.34	1.25	12561	100
	GALS	86.5	2.06	29209.4	80
	ABC(I)-ELM	89.01	1.02	1658.5	410
	ABC(II)-ELM	89.17	0.98	1499.3	400
	IWO(I)-ELM	89.10	0.99	1470.2	400
	IWO(II)-ELM	89.02	1.03	3523.4	410
Shuttle	E-ELM	99.53	0.12	1336.9	20
	ELM	99.51	0.1	2.8206	200
	DE-LM	99.33	0.09	10892.3	20
	LM	99.27	0.13	6132.2	50
	GALS	Out of memory			
	ABC(I)-ELM	99.54	0.08	6486.8	160
	ABC(II)-ELM	99.53	0.08	5799.5	150
	IWO(I)-ELM	99.57	0.08	5716.6	150
	IWO(II)-ELM	99.54	0.08	14321	160

[25] consists of the measurements of three types of flowers, viz. Setosa, Verginica and Versicolor. There are three classes in this data set, each having 50 patterns. Wine data is obtained from a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars. The data set consists of 178 patterns belonging to three types of wines. Image segmentation data set consists of seven outdoor images. The images are brickface, sky, foliage, cement, window, path and grass. The images were hand-segmented to create a classification for every pixel. Satellite image data set consists of the multi-spectral values of pixels in 3×3 neighborhoods in a satellite image, and the classification associated with the central pixel in each neighborhood. Shuttle data set deals with the positioning of radiators in the Space Shuttle and consists of 58000 patterns belonging to seven different classes. Approximately 80% of the data belongs to one of the seven classes. Cardiotocography data set consists of measurements of fetal

Table 5 Classification performance of ELM and the proposed ABC-ELM and IWO-ELM algorithms on five additional data sets

Data sets	Methods	Testing (%)		Training time (sec.)	Neurons
		Accuracy	Dev		
Cardiotocography-3	ELM	91.26	1.1	0.3221	390
	ABC(I)-ELM	91.36	1.28	166.163	175
	ABC(II)-ELM	91.45	1.21	185.334	195
	IWO(I)-ELM	91.36	1.34	152.324	160
	IWO(II)-ELM	91.39	1.17	435.424	200
Cardiotocography-10	ELM	81.3	1.58	0.3627	400
	ABC(I)-ELM	81.36	1.46	173.26	180
	ABC(II)-ELM	81.92	1.49	193.507	200
	IWO(I)-ELM	81.92	1.48	181.44	190
	IWO(II)-ELM	81.17	1.72	399.568	195
Phishing websites	ELM	93.12	1.16	0.2104	300
	ABC(I)-ELM	93.21	0.96	134.122	125
	ABC(II)-ELM	93.12	1.1	121.098	120
	IWO(I)-ELM	93.52	0.91	107.031	100
	IWO(II)-ELM	93.06	0.94	287.421	125
Vehicle	ELM	82.5	2.89	0.0667	220
	ABC(I)-ELM	83.07	3.02	31.9926	90
	ABC(II)-ELM	82.8	3.1	35.7024	100
	IWO(I)-ELM	83.03	2.75	30.6404	95
	IWO(II)-ELM	82.5	2.68	72.5376	95
Wine quality	ELM	60.68	2.08	0.0763	200
	ABC(I)-ELM	60.76	2.22	68.6087	105
	ABC(II)-ELM	61.01	2.2	66.8942	105
	IWO(I)-ELM	61.09	2.03	50.6773	90
	IWO(II)-ELM	60.6	2.28	113.329	90

heart rate (FHR) and uterine contraction (UC) features on cardiotocograms classified by expert obstetricians. Classification was both with respect to a morphologic pattern (10 classes; 1–10) and to a fetal state (three classes; normal, suspect and pathologic). Therefore the data set can be used either for 10-class (Cardiotocography-10) or 3-class (Cardiotocography-3) experiments. Phishing websites data set consists of 30 important features that have proved to be sound and effective in predicting phishing websites. The purpose of Vehicle data set is to classify a given silhouette as one of four types of vehicles, using a set of features extracted from the silhouette. Wine quality data set is related to red variant of the Portuguese "Vinho Verde" wine. The goal is to model wine quality based on physiochemical tests.

5.2 Results and discussion

The proposed ABC-ELM and IWO-ELM algorithms are tested on twelve benchmark classification data sets as mentioned already. The ABC-ELM and IWO-ELM algorithms run 50 times on each of these data sets and the average results are presented in Tables 3, 4 and 5. For each run of simulations, the training data set and testing data set are randomly generated according to the specifications given in Tables 1 and 2. We have used the sigmoid function as an activation function in ELM hidden layer. The initial solutions of ABC-ELM and IWO-ELM algorithms are randomly generated over the range [−1, 1]. For ABC-ELM, the number of food sources $SN = 10$, the number of onlookers is 10 and the $limit = 10$. For IWO-ELM, the number of initial solutions $WN_{init} = 5$, the maximum number of solutions allowed in the colony $WN_{max} = 10$, $S_{max} = 5$ and $S_{min} = 0$. For both ABC-ELM and IWO-ELM, we set $nmi = 3$, $\sigma_{initial} = 0.8$, $\sigma_{final} = 0.1$, $Chg_{max} = 20$, $Chg_{min} = 3$, $P_{initial} = 0$ and $P_{final} = 1$. Both ABC-ELM and IWO-ELM algorithms terminate if the maximum number of iterations $iter_{max} = 100$ is reached. All the simulations are carried out in MATLAB environment on a system with Core i5-2400, 3.10 GHZ CPU and 4 GB RAM.

The performance of ABC-ELM and IWO-ELM algorithms on the first four classification data sets in Table 1 has been compared with the techniques given in [13], whereas the comparison on the remaining three data sets in this table is done with the methods given in [12]. Actually, first four classification data sets in Table 1 were used

in [13] to test the performance of various approaches proposed. On the other hand, [12] used the remaining three data sets in this table. Tables 3 and 4 show the classification performance of the proposed approaches along with the methods presented in [12, 13] respectively. Table 5 shows the classification performance of ELM and the proposed ABC-ELM and IWO-ELM algorithms on data sets with relatively medium size and medium dimensions. For ABC-ELM and IWO-ELM algorithms, similar to the simulations in [12, 13], the number of hidden neurons is gradually increased and the one with the best testing accuracy is reported in Tables 3, 4 and 5.

In Table 3, we report the best, average and worst performance of E-ELM and DE-LM over eight different combinations of generation strategies and their control parameters as specified in [13]. From Table 3, we can see that IWO-ELM obtains the best classification performance on Iris and Wine data sets while ABC-ELM is the second best on these two data sets. Both ABC-ELM and IWO-ELM achieve good generalization performance on Disease data set with less number of neurons. SaE-ELM obtains the best testing accuracy on Diabetes and Disease data sets and both ABC-ELM and IWO-ELM achieve satisfying results on Diabetes data set with small number of neurons. Both ABC-ELM and IWO-ELM outperform the average performance of E-ELM and DE-LM on all the data sets. We can observe that the training time of ABC-ELM and IWO-ELM is higher than that of the other approaches except for SVM, and CFWNN-ELM is the fastest. From Table 3, we can also observe the standard deviation of ABC-ELM and IWO-ELM is small compared to other methods. IWO-ELM outperforms ABC-ELM on all the data sets, but ABC-ELM obtains comparable results on these data sets with less training time. Both ABC-ELM and IWO-ELM achieve better results with the second neighborhood procedure (i.e. Algorithm 3) than that with the first neighborhood procedure (i.e. Algorithm 2).

The data sets in Table 3 are relatively small and the number of classes at maximum equals to 3. The data sets used in [12] are relatively large and the number of classes is high. We have compared the performance of ABC-ELM and IWO-ELM on these data sets with the methods presented in [12] and the results are reported in Table 4. The results of our implementation of ELM on these data sets are reported in Table 4. From Table 4, we observe that both ABC-ELM and IWO-ELM outperform the other algorithms on all the data sets. IWO-ELM obtains better results than ABC-ELM on Image segmentation and Shuttle data sets. We also observe that IWO-ELM achieves better results with the first neighborhood procedure than that with the second neighborhood procedure. Compared to ELM, the integration of ABC and IWO helps to relatively reduce the number of hidden neurons in ABC-ELM and IWO-ELM, respectively and also helps to improve the generalization performance. From Table 4, we can see that the training time of ELM is much less than that of other approaches. We can also observe that the training time of IWO(II)-ELM is higher than that of IWO(I)-ELM, ABC(I)-ELM and ABC(II)-ELM. E-ELM obtains good generalization performance, but its training time is relatively high even though the number of neurons is small.

To further investigate the applicability of the proposed approaches, five benchmark data sets of medium size and dimensions have been used to test the performance of ELM, ABC-ELM and IWO-ELM. The results of the comparison are shown in Table 5. From Table 5, we can observe that the proposed ABC-ELM and IWO-ELM obtain higher generalization performance than the original ELM in most cases. The results show that ABC and IWO not only help to reduce the number of hidden neurons in ABC-ELM and IWO-ELM respectively, but also help to improve the generalization performance. While ABC-ELM obtains better results than IWO-ELM on Cardiotocography-3 and Vehicle data sets, IWO-ELM obtains the best results on Phishing websites and Wine quality data sets. IWO-ELM obtains the same result as ABC-ELM on Cardiotocography-10 data set, but with less number of neurons. From Table 5, we can observe that even though the number of neurons are relatively different, the training times of ABC(I)-ELM, ABC(II)-ELM and IWO(I)-ELM are small and there is not much difference. However, despite the fact that IWO(II)-ELM obtains similar performance with relatively same number of neurons as compared to ABC(I)-ELM, ABC(II)-ELM and IWO(I)-ELM, it seems to be the use of second neighborhood procedure (i.e. Algorithm 3) in IWO(II)-ELM prolongs the training process which requires further investigations.

6 Conclusion

In this paper, we have proposed two metaheuristic approaches for training single-hidden layer feedforward neural networks. The two approaches (referred to as ABC-ELM and IWO-ELM) use artificial bee colony algorithm (ABC) and Invasive Weed Optimization algorithm (IWO) to tune the network input weights and hidden biases respectively, and extreme learning machine (ELM) to determine the output weights analytically. Both ABC and IWO algorithms are simple, robust and easy to understand and implement. The performance of proposed ABC-ELM and IWO-ELM have been evaluated on twelve benchmark classification data sets. Their performance is also compared with state-of-the-art techniques available in the literature. Experimental results show that the

proposed approaches achieve good generalization performance in comparison to other techniques. Our approaches are the first approaches based on swarm intelligence to tune the ELM hidden parameters. Hence, they will serve as the baseline approaches for any future swarm intelligence based approach for tuning ELM hidden parameters.

There are several issues remaining which provide scopes for future studies such as developing an efficient way to reduce the training time required by ABC-ELM and IWO-ELM. A possible approach in this direction will be to use the methods proposed in [26] for computing the Moore-Penrose generalized inverse efficiently. There is also scope for developing additional neighborhood procedures in such a way that they can be used in a self-adaptive manner on the lines of SaE-ELM [13] and develop what can be called self-adaptive ABC-ELM and self-adaptive IWO-ELM. Another possible future work will be to study the performance of our approaches in combination with other methods such as those proposed in [27–29].

References

1. Huang GB, Zhu QY, Siew CK (2004) Extreme learning machine: A new learning scheme of feedforward neural networks. In: Proceedings of International Joint Conference on Neural Networks (IJCNN), vol 2, Budapest, Hungary, pp 985–990
2. Huang GB, Zhu QY, Siew CK (2006) Extreme learning machine: theory and applications. Neurocomputing 70:489–501
3. Huang GB, Chen L, Siew CK (2006) Universal approximation using incremental constructive feedforward networks with random hidden nodes. IEEE Trans Neural Netw 17(4):879–892
4. Huang GB, Chen L (2007) Convex incremental extreme learning machine. Neurocomputing 70:3056–3062
5. Cao J, Chen T, Fan J (2016) Landmark recognition with compact bow histogram and ensemble elm. Multimed Tools Appl 75(5):2839–2857
6. Cao J, Zhang K, Luo M, Yin C, Lai X (2016) Extreme learning machine and adaptive sparse representation for image classification. Neural Netw 81:91–102
7. Serre D (2002) Matrices: theory and applications. Springer, NewYork
8. Huang GB, Zhou H, Ding X, Zhang R (2012) Extreme learning machine for regression and multiclass classification. IEEE Trans Syst Man Cybern B Cybern 42(2):513–529
9. Rong HJ, Ong YS, Tan AH, Zhu Z (2008) A fast pruned-extreme learning machine for classification problem. Neurocomputing 72:359–366
10. Miche Y, Sorjamaa A, Bas P, Simula O, Jutten C, Lendasse A (2010) OP-ELM: optimally pruned extreme learning machine. IEEE Trans Neural Netw 21(1):158–162
11. Lan Y, Soh YC, Huang GB (2010) Constructive hidden nodes selection of extreme learning machine for regression. Neurocomputing 73:3191–3199
12. Zhu QY, Qin A, Suganthan P, Huang GB (2005) Evolutionary extreme learning machine. Pattern Recogn 38:1759–1763
13. Cao J, Lin Z, Huang GB (2012) Self-adaptive evolutionary extreme learning machine. Neural Process Lett 36:285–305
14. Karaboga D (2005) An idea based on honey bee swarm for numerical optimization. technical report tr06, Computer Engineering Department. Erciyes University, Turkey
15. Mehrabiana A, Lucas C (2006) A novel numerical optimization algorithm inspired from weed colonization. Ecol Inform 1:355–366
16. Akay B, Karaboga D (2012) A modified artificial bee colony algorithm for real-parameter optimization. Inf Sci 192:120–142
17. Basak A, Pal S, Das S, Abraham A, Snasel V (2010) A modified invasive weed optimization algorithm for time-modulated linear antenna array synthesis. In: Proceedings of IEEE Congress on Evolutionary Computation (CEC). Barcelona, pp 1–8
18. Broomhead DS, Lowe D (1988) Multivariable functional interpolation and adaptive networks. Complex Syst 2(3):321–355
19. Schmidt WF, Kraaijveld MA, Duin RP (1992) Feedforward neural networks with random weights. In: Proceedings of 11th IAPR International Conference on Pattern Recognition Methodology and Systems. Hague, Netherlands, pp 1–4
20. Pao YH, Park GH, Sobajic DJ (1994) Learning and generalization characteristics of random vector functional-link net. Neurocomputing 6:163–180
21. Huang GB (2003) Learning capability and storage capacity of two-hidden-layer feedforward networks. IEEE Trans Neural Netw 14(2):274–281
22. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. J Glob Optim 39(3):459–471
23. Pandiri V, Singh A (2015) Two metaheuristic approaches for the multiple traveling salesperson problem. Appl Soft Comput 26:74–89
24. Zhou Y, Luo Q, Chen H, He A, Wu J (2015) A discrete invasive weed optimization algorithm for solving traveling salesman problem. Neurocomputing 151:1227–1236
25. Fisher RA (1936) The use of multiple measurements in taxonomic problems. Ann Eugen 7:179–188
26. Lu S, Wang XZ, Zhanga G, Zhoua X (2015) Effective algorithms of the moore-penrose inverse matrices for extreme learning machine. Intell Data Anal 19:743–760
27. Ashfaq RAR, Wang XZ, Huang JZ, Abbas H, He YL (2017) Fuzziness based semi-supervised learning approach for intrusion detection system. Inf Sci 378:484–497
28. Wang XZ, Aamir R, Fu AM (2015) Fuzziness based sample categorization for classifier performance improvement. J Intell Fuzzy Syst 29:1185–1196
29. Wang XZ, Xing HJ, Li Y, Hua Q, Dong CR, Pedrycz W (2015) A study on relationship between generalization abilities and fuzziness of base classifiers in ensemble learning. IEEE Trans Fuzzy Syst 23:1638–1654